

# COSC 39: Theory of Computation

**Credit Statement:** Talked to Sair Shaikh'26.

**Problem 1. Redundant code detector.**

Imagine there is a way to automatically check if some of your codes are redundant, as in that there are no inputs that will ever lead to certain part of the code being run. How convenient is that!

Alas, life is not that simple. We can intuitively view a single-line instruction / code as a state in the Turing machine. Call a state  $q$  in a Turing machine useless if no input string would ever lead to the TM entering  $q$ .

Prove that the following language is undecidable.

$$\text{USELESS?} := \{ \langle M \rangle \mid \text{Turing machine } M \text{ has a useless state} \}$$

*Solution.*

Assume for contradiction that some Turing machine  $M_U$  decides USELESS. We will reduce the un-computable problem,

$$\text{ACCCEPT} := \{ (\langle M \rangle, w) \mid \text{TURING MACHINE } M \text{ ACCEPTS } w \}$$

to USELESS. Construct a wrapper function WRAP around  $M_U$  such that given an input  $(\langle M \rangle, w)$ , WRAP behaves as follows:

Modify the code  $\langle M \rangle$  into code  $\langle M' \rangle$ , which has added state  $q^*$  and behaves as follows:

On input string  $x$ :

1. simulate  $M$  on  $w$ .
2. If  $M$  accepts  $w$ , go to defined state  $q^*$  and accept
3. If  $M$  doesn't accept  $w$ , reject

Where  $q^*$  is a new defined state that is only entered upon the condition in step 2.  $M'$ 's states, which are used to simulate  $M$ , are reachable because the simulator (which has states like a UTM) runs no matter what on every input to simulate  $M$ .  $M'$ 's states are just recorded on the tape, so even if  $M$  has an unreachable state, it's not a state in  $M'$ . Also note that  $M'$  ignores its input  $x$  and only simulates  $M$  on the  $w$ , i.e. the reachability of  $q^*$  is uniform across all inputs.

Notably, we never execute  $M'$  in WRAP; we only construct the code  $\langle M' \rangle$ . We then feed  $\langle M' \rangle$  as the input into  $M_U$  and return the flipped result of  $M_U(\langle M' \rangle)$ . If  $M_U$  rejects (there are no useless states), we accept in WRAP. If  $M_U$  accepts (there is a useless state), we reject in WRAP.

Notably, the states which  $M'$  can reach depend on whether  $M$  accepts  $w$ . If  $M$  accepts  $w$ ,  $M'$  goes to  $q^*$ . If  $M$  does not accept  $w$ ,  $M'$  does not reach  $q^*$ .

1. Given a yes-instance  $\langle M' \rangle$  of ACCEPT,  $M$  accepts  $w$ . Thus on input  $x$ ,  $M'$  goes to  $q^*$ . Because  $M'$  has no useless states,  $M_U$  rejects  $\langle M' \rangle$ , and thus it is a no-instance of USELESS.
2. Given a no-instance  $\langle M' \rangle$  of ACCEPT,  $M$  does not accept  $w$ . Thus on input  $x$ ,  $M'$  does not go to  $q^*$ . Because  $M'$  has a useless state  $q^*$ ,  $M_U$  accepts  $\langle M' \rangle$ , and thus it is a yes-instance of USELESS.

By simply returning the flipped output of  $M_U$ , WRAP solves ACCEPT, which is a contradiction. Consequently, no such  $M_U$  can exist that decides USELESS.

## Problem 2. Self-referential machines.

- (a) Design an algorithm  $B$  that, assuming that its own source code  $\langle B \rangle$  is conveniently given as input when starts, outputs the source code of another algorithm  $A$  that prints the source code of  $B$  on execution. (The algorithm should be described using pseudocode, not an explicit Turing machine.)
- (b) Design an algorithm that outputs its own source code without any input. (Remember that your algorithm does not have access to its own source code. The executable of the algorithm alone has to generate its source code back.)

[Hint: As a warm-up, can you construct a Turing machine  $M$  that takes no input, and output a sequence of 1s on the tape, where the number of 1s is equal to two times the number of transitions (edges) in  $M$ ? Then, think about how to use (a).]

*Solution.*

- (a) We define the algorithm  $B$  as follows:

```
B(x):  
    print("print("+ x + ")")
```

We will refer to the output of the algorithm  $B$  as  $\langle A_x \rangle$  where  $x$  is the input variable.

```
A_x:  
    print(x)
```

Then, running  $B$  on it's own source code, i.e.  $x = \langle B \rangle$  we have,  $\langle A_{\langle B \rangle} \rangle = \text{print}(\langle B \rangle)$ . Note that executing  $A_{\langle B \rangle}$  will give us  $\langle B \rangle$ .

- (b) For this part, we construct the algorithm  $Q$  which returns its own source code upon execution without any input. To do so, we modify our algorithm  $B$  from part (a) as follows:

```
B(x):  
    print("print("+ x + ") + x")
```

As programmers, we have defined two algorithms:  $A_x()$  and  $B(x)$ . Since we have the source code for both the algorithms, we can execute  $B(\langle B \rangle)$  and get back  $\langle A_{\langle B \rangle} \rangle \cdot \langle B \rangle$ . Then, we can extract  $A_{\langle B \rangle}$  from the output and copy this over in our definition of the algorithm  $Q$ .

$Q$ :

```
 $A_{\langle B \rangle}$   
def  $B(x)$  :  
    print("print("+  $x$  + ")" +  $x$ )
```

$Q$  executes in 2 parts.

- (a) First, it executes  $A_{\langle B \rangle}$ , writing  $\langle B \rangle$  to a work tape.
- (b) Then, it reads  $\langle B \rangle$  from the work tape as input (i.e.  $x = \langle B \rangle$ ) and outputs

$$B(\langle B \rangle) = A_{\langle B \rangle} \cdot \langle B \rangle$$

which is exactly the source code of  $Q$ . Thus,  $Q$  outputs  $\langle Q \rangle$ .